

ATOM Intelligent Recommendation Engine

System Architecture, Pipeline Flow & Modular Production Specification

Architecture: 13 Sequential Modules

Target: Production Kiosk & Web App

Verification: 100% Automated Unit & Live Coverage

Date: October 2026

1. Executive Summary & Philosophy

The ATOM Intelligent Recommendation Engine is a deterministic, multi-stage recommender architecture built specifically for Quick Service Restaurant (QSR) kiosks. Unlike black-box machine learning approaches that hallucinate unavailable combinations or violate dietary constraints, ATOM uses a **guaranteed override hierarchy** paired with sensory heuristics, basket saturation analysis, and diversity re-ranking.

100% Deterministic Safety

Dietary locks (Veg/Non-Veg), out-of-stock items, price limits, and items already in the cart are hard-filtered before any scoring can occur. AI never decides what is available or legally compliant.

Mutual Exclusion Slicing

Items are partitioned into **Priority (top 2)**, **Premium (top 2)**, and **Additional (next 4)** without duplicate cards across shelves.

Omni-Channel Session Memory

Dietary choices made on Burgers persist automatically across Sides, Drinks, and Desserts without leaking opposing food types.

Whitecard & Refresh Immunity

Root-relative asset paths and instant fallback data fetchers prevent blank cards and 404 errors during direct navigation or browser refreshes.

2. End-to-End Recommendation Pipeline Flow

When a user views a category, adds an item, or navigates the kiosk, recommendations pass through 5 strict sequential stages:

1 Stage 1: Session Context & Dietary Gate (Module 1)

Reads the user's active dietary lock from `conversation_context["food_preference"]`. Eliminates opposing food items at both the database query level (`menu_repository.py`) and in-memory module (`m01_dietary_lock.py`).

2 Stage 2: Cart Deduplication & Host Gating (Modules 6 & 7)

Removes all items already present in the user's basket (`m07_cart_exclusion.py`). Suppresses dipping sauces unless finger-food host items like Fries or Nuggets are present (`m06_condiment_gating.py`).

3 Stage 3: Base Scoring & Urgency Multipliers

Calculates base score from stock levels and days-to-expiry (`get_priority_items`). High-inventory or near-expiry items receive urgency points to prevent commercial food waste.

4

Stage 4: Neuro-Gastronomic & Commercial Intelligence (Modules 2, 3, 5, 9, 10, 11, 12)

Applies mathematical multipliers: **Sensory Contrast (1.35x)** for spicy mains paired with cooling dairy; **Anti-Redundancy (0.35x)** to prevent potato-on-potato fatigue; **Flavor Synergy (1.50x)** for regional pairings; **Circadian Craving (1.20x - 1.25x)** for time-of-day boosts; **Margin Multiplier (1.10x - 1.30x)** for high-margin fountain drinks.

5

Stage 5: Basket Saturation & Sub-Role MMR Diversity (Modules 4, 8, 13)

Evaluates the 4 dining pillars (Main, Side, Drink, Dessert). If all pillars are fulfilled, the checkout killswitch suppresses tray clutter (`m08_gatekeeper_killswitch`). Otherwise, two-pass sub-role MMR (`m13_subrole_mmr`) ensures items displayed are diverse (e.g. never 3 thick shakes together).

3. Comprehensive Module Master Catalog (M1 - M13)

Every recommendation capability is isolated into a standalone, sequentially integrable module in

`MIGRATION/MODULES/` with dedicated specifications, unit tests, and drop-in integration files:

Module	File	Type	Description & Commercial Impact
M1: Dietary Lock	<code>m01_dietary_lock.py</code>	Hard Gate	Strict Veg/Non-Veg lock across all menus and trays. Eliminates non-veg leaks. Normalizes root image paths (whitecard fix) and provides F5 refresh fallbacks.
M2: Anti-Redundancy	<code>m02_anti_redundancy.py</code>	Penalty	Dampens duplicate base ingredients (e.g. Potato burger + Hashbrown receives 0.35x penalty). Universal fries bypass penalty.
M3: Flavor Synergy	<code>m03_flavor_synergy.py</code>	Multiplier	0.25x anti-clash dampener (Mango + Mango clash) and 1.50x regional synergy boost (Makhani burger + Masala fries).
M4: Dual-Role Saturation	<code>m04_dual_role_saturation.py</code>	Gate	Tracks Main/Side/Drink/Dessert dining pillars. Recognizes hybrid items (Thick Shake fulfills both Drink + Dessert). Suppresses saturated categories.
M5: Budget Ceiling	<code>m05_budget_ceiling.py</code>	Ceiling	Drops companion cross-sell suggestions priced greater than <code>anchor_price * 1.5</code> to protect customer price-sensitivity.
M6: Condiment Gating	<code>m06_condiment_gating.py</code>	Gate	Suppresses standalone dips and sauces unless a finger-food host item (Fries, Nuggets) is in the basket. Boosts dips by 1.5x when host is present.
M7: Cart Exclusion	<code>m07_cart_exclusion.py</code>	Hard Gate	Filters out items already added to the cart by ID and normalized name. Eliminates duplicate product card spam on recommendation trays.
M8: Gatekeeper Killswitch	<code>m08_gatekeeper_killswitch.py</code>	Hard Gate	Suppresses recommendation shelves at checkout when basket fulfills all 4 dining pillars, accelerating checkout conversion without friction.
M9: Margin Multiplier	<code>m09_margin_multiplier.py</code>	Commercial	1.10x–1.30x score boost for high-margin fountain beverages scaled by basket size; 0.85x dampener for low-margin packaged water bottles.
M10: Sensory Contrast	<code>m10_sensory_contrast.py</code>	Gastronomy	Neuro-gastronomic pairing: Spicy Main -> Cooling Dairy (1.35x boost); Heavy/Cheesy Main -> Carbonated Cut-Through Beverage (1.30x boost).
M11: Session Fatigue	<code>m11_session_fatigue.py</code>	Novelty	Dampens items viewed $\geq 2x$ (0.85x) and $\geq 3x$ (0.70x) within the same session, bubbling up fresh catalog items to combat banner blindness.
M12: Circadian Craving	<code>m12_circadian_craving.py</code>	Timing	Time-of-day boosts: Morning coffee/breakfast (1.25x), Lunch hearty burgers (1.20x), Evening snack sides (1.20x), Late-night desserts (1.25x).

Module	File	Type	Description & Commercial Impact
M13: Sub-Role MMR	m13_subrole_mmr.py	Diversity	Two-pass Maximal Marginal Relevance. Prevents category monopolies (never two shakes or two fries together). Guarantees offer price integrity.

New Inventions & Future Modules Protocol

Whenever a new recommendation module or refinement is created:

- 1. Create isolated logic in `original/MIGRATION/MODULES/M<N>_...` with `SPEC.md` , `files/` , `merge/` , and self-contained `tests/` .
- 2. Apply the exact change to `modules/` (`Intelligent-agent-clean-master`).
- 3. Run the universal test suite to verify 0 regressions across all modules.
- 4. Update this living Architecture Report to document the new capability.

4. Operational Verification & Guarantees

- **100% Automated Unit Test Suite:** Every module features an isolated test script with zero external dependencies, running in sub-second time.
- **Database-to-UI Integrity:** All menu items are normalized with leading slashes, pre-filtered by preference in SQL, and protected by image error fallbacks in React.
- **Zero Ghost Dependencies:** Audio and voice libraries (Whisper, Kokoro) degrade gracefully when hardware/drivers are not present, ensuring the server starts instantly in any environment.